

Advanced Compilers

Project 4

Virtual Machine & Optimization Laboratory
Dept. of Electrical and Computer Engineering
Seoul National University



Index

- 1. Vibe Coding for Implementing LLVM Passes**
- 2. Value substitution**
- 3. Project 4**



VIBE CODING FOR IMPLEMENTING LLVM PASSES

Vibe Coding

Vibe Coding

- 엄밀한 논리나 구체적인 설계 없이, 일상적인 언어(자연어)로 아이디어나 느낌을 전달하면 AI가 알아서 코드를 생성 및 수정해 주는 소프트웨어 개발 방식
- 다양한 개발사가 제공하는 Code Agent CLI 환경 혹은 IDE 환경을 활용

Gemini

deepseek

OpenAI
BENEFITS ALL OF HUMANITY



GitHub
Copilot

Claude



CURSOR



TRAE



replit



Windsurf



Gemini CLI

Google의 Coding Agent

- AI Agent가 CLI 환경에서 codebase에 직접 접근
- 코드 생성, 수정, 실행까지 자동으로 수행
- `npm install -g @google/gemini-cli`

Build  debug
& deploy with AI

Query and edit large codebases, generate apps from images or PDFs, and automate complex workflows—all from your terminal with Gemini 3.

```
$ npm install -g @google/gemini-cli
```

Gemini CLI 설치

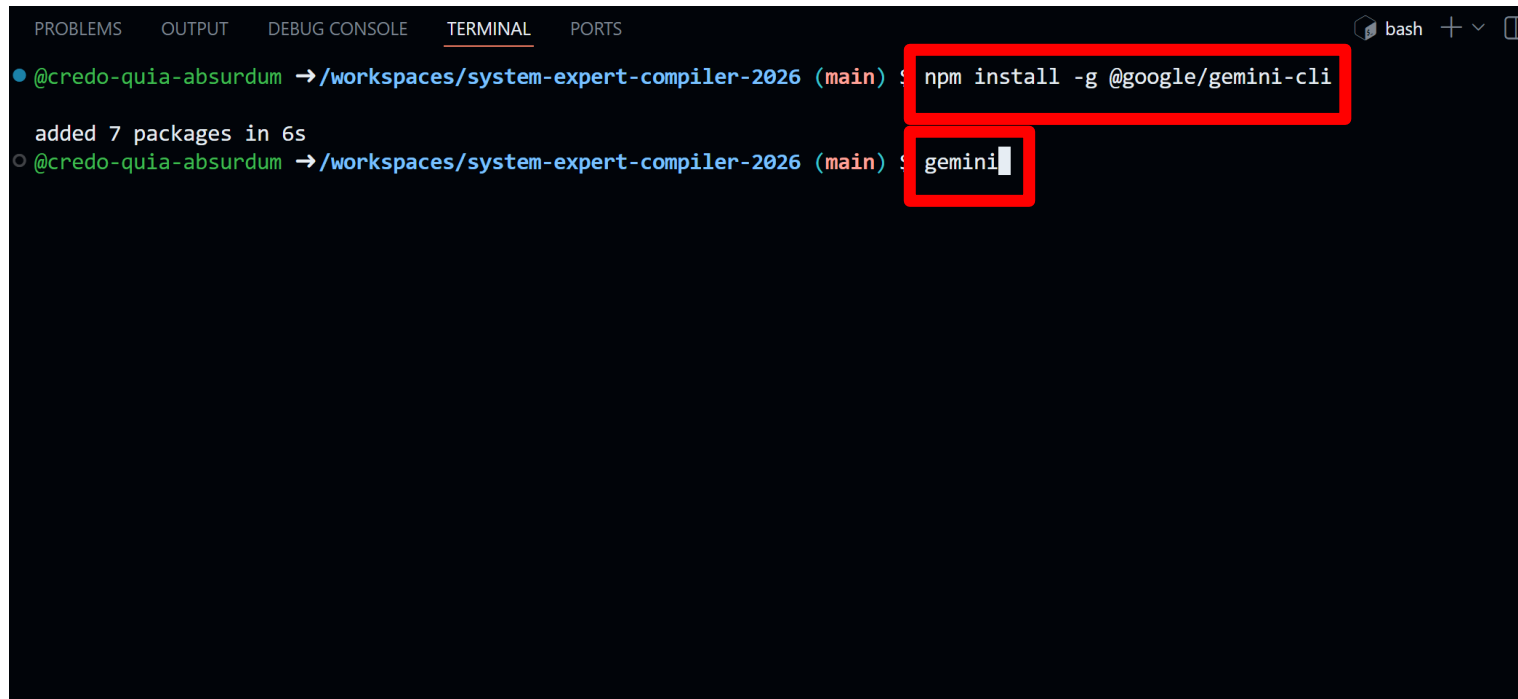
설치 명령어

➤ `npm install -g @google/gemini-cli`

실행 명령어

➤ Working Directory로 이동하여 실행

➤ `gemini`



The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows the following commands and output:

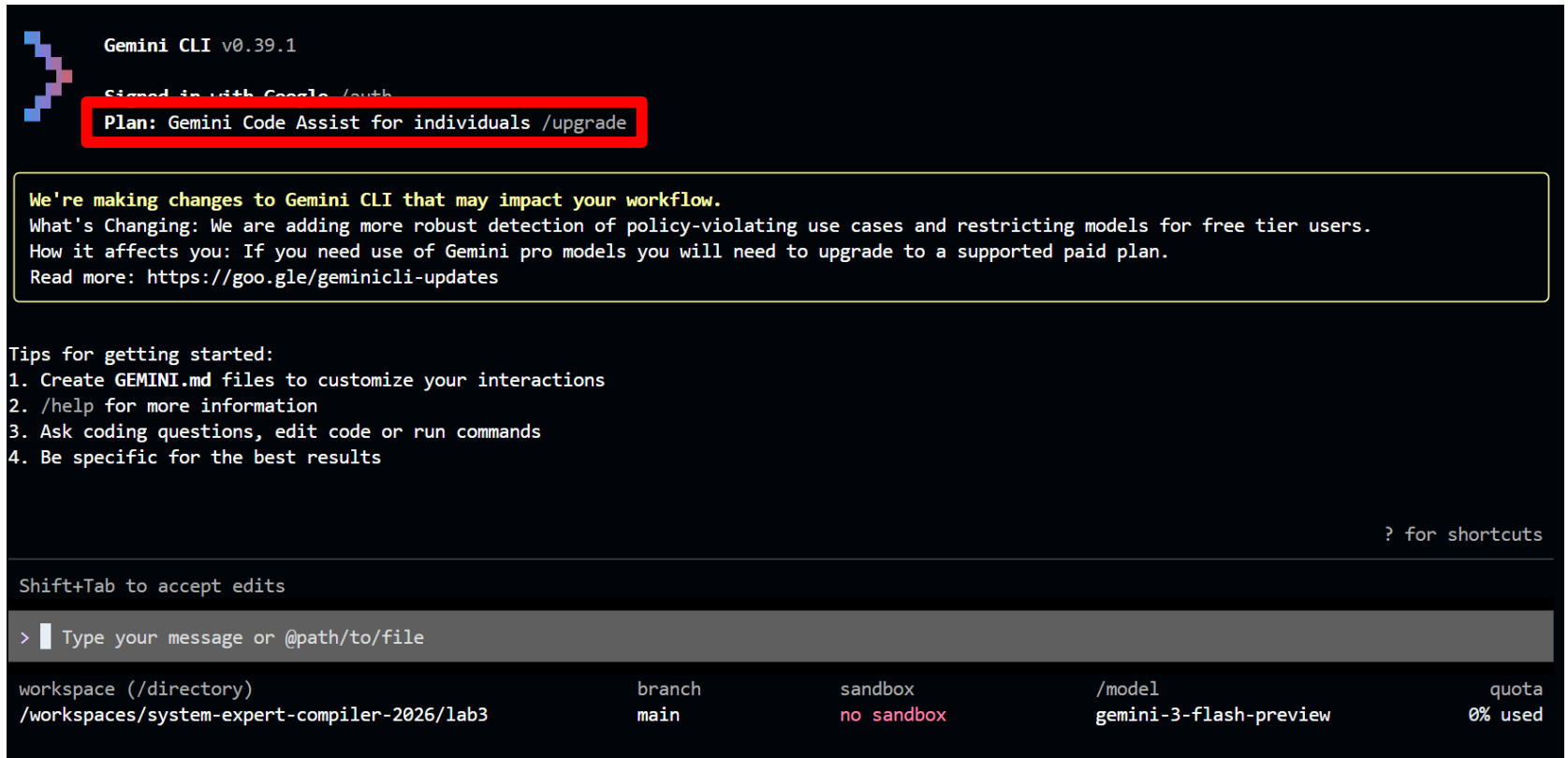
```
@credo-quia-absurdum → /workspaces/system-expert-compiler-2026 (main) $ npm install -g @google/gemini-cli
added 7 packages in 6s
@credo-quia-absurdum → /workspaces/system-expert-compiler-2026 (main) $ gemini
```

The command `npm install -g @google/gemini-cli` is highlighted with a red box, and the command `gemini` is also highlighted with a red box.

Gemini CLI

세팅 완료 후의 화면

➤ Gemini Code Assist for Individuals (Free-Tier)



```
Gemini CLI v0.39.1
Signed in with Google /auth
Plan: Gemini Code Assist for individuals /upgrade

We're making changes to Gemini CLI that may impact your workflow.
What's Changing: We are adding more robust detection of policy-violating use cases and restricting models for free tier users.
How it affects you: If you need use of Gemini pro models you will need to upgrade to a supported paid plan.
Read more: https://goo.gle/geminicli-updates

Tips for getting started:
1. Create GEMINI.md files to customize your interactions
2. /help for more information
3. Ask coding questions, edit code or run commands
4. Be specific for the best results

? for shortcuts

Shift+Tab to accept edits
> | Type your message or @path/to/file

workspace (/directory)      branch      sandbox      /model      quota
/workspaces/system-expert-compiler-2026/lab3    main        no sandbox    gemini-3-flash-preview    0% used
```

Gemini CLI - Commands

주요 명령어

- `/help`
- `/quit` - 세션 종료
- `/resume` - 이전 세션에 이어서 시작
- `/plan` - 수정 없이 계획만 논의
- `/model` - 사용할 LLM 모델 변경
- `/init` - 프로젝트 구조를 분석하고 맞춤형 GEMINI.md 생성
 - GEMINI.md
 - 프로젝트 이해를 위한 context file
 - 코드 구조, 규칙, 의존성 등을 정리하여 AI agent가 더 정확하게 작업할 수 있도록 함
- `/compress` - Context를 압축하여 요약으로 교체
 - LLM의 Context Length 제약 사항을 초과하는 작업을 이어나갈 때 유용
 - 명시적으로 호출하지 않아도 Context Length Limit에서 자동으로 호출 됨
- `/auth` - google 계정 로그인 관련 명령 수행

Practice with Code Agent

/init

- 프로젝트 전체를 살펴보고, 관련 내용을 정리함
- 이후의 작업에서 가장 기초적으로 참고하는 내용이 됨

```
> /init
```

```
i Empty GEMINI.md created. Now analyzing the project to populate it.
```

```
✓ ReadFile CMakeLists.txt
✓ ReadFile reference_commands.txt
✓ ReadFile lib/CMakeLists.txt
✓ ReadFile include/InstructionCounter.h
✓ ReadFile lib/CMakeLists.txt
✓ WriteFile GEMINI.md → Accepted (+35, -0)
```

```
1 # Project Overview
2
3 This project is an LLVM-based analysis pass implementation for the Advanced Compilers
  lab. It provides an `InstructionCounter` pass designed to analyze LLVM IR and count
  instructions within functions.
4
5 # Building and Running
6
7 ## Building the project
8 Configure and build using `cmake` and `ninja`:
9
10 ```bash
11 cmake -S . -G Ninja -B build
12 cmake --build build
13 ```
14
15 ## Running the analysis pass
16 The project generates an LLVM pass plugin (`build/lib/libInstructionCounter.so`). To run
  it on a C file:
17
```

Project Overview

This project is an LLVM-based analysis pass implementation for the Advanced Compilers lab. It provides an `InstructionCounter` pass designed to analyze LLVM IR and count instructions within functions.

Building and Running

Building the project

Configure and build using `cmake` and `ninja`:

```
```bash
cmake -S . -G Ninja -B build
cmake --build build
```
```

Running the analysis pass

The project generates an LLVM pass plugin (`build/lib/libInstructionCounter.so`). To run it on a C file:

1. Generate LLVM IR from the source code:

```
```bash
clang -S -O0 -emit-llvm -Xclang -disable-O0-optnone
-fno-discard-value-names test.c -o test.ll
```
```

2. Run the `InstructionCounter` pass using `opt`:

```
```bash
opt -S -load-pass-plugin='build/lib/libInstructionCounter.so'
-passes='instruction-counter' test.ll -disable-output 2> test.out
```
```

Practice with Code Agent

구현할 내용이 간단하고 GEMINI.md에서 잘 파악되어 있으므로 프롬프트로 구현 명령

♦ I have successfully analyzed the directory and generated the GEMINI.md file.

> 한번 구현해 볼래?

♦ 네, `InstructionCounter` LLVM 패스의 할당 부분을 구현해 보겠습니다.

GEMINI.md 파일과 `lib/InstructionCounter.cpp` 파일을 기반으로, `run` 메소드 내의 `***** ASSIGNMENT *****` 주석 섹션에 각 명령어(`instruction`)의 `opcode` 이름을 기반으로 명령어 수를 세는 로직을 추가하겠습니다.

다음 단계를 따르겠습니다:

1. `lib/InstructionCounter.cpp` 파일을 읽어 수정할 부분을 정확히 찾습니다.
2. `run` 메소드 안에서 함수의 각 기본 블록(`BasicBlock`)과 그 안의 명령어들을 순회하는 로직을 추가합니다.
3. 각 명령어에 대해 `opcode` 이름을 가져와 `InstructionCounter` 맵에 해당 명령어의 수를 증가시킵니다.
4. 수정된 내용을 `lib/InstructionCounter.cpp` 파일에 다시 작성합니다.

✓ Edit `InstructionCounter.cpp` → Accepted (+6, -6)

```
10 PreservedAnalyses
    InstructionCounter::run(Function &F,
11
12     //***** ASSIGNMENT
    *****
13 -
14 -
15 -
16 -
17 -
18 -
13 +   for (auto &BB : F) {
14 +       for (auto &Inst : BB) {
15 +          StringRef OpcodeName =
               Inst.getOpcodeName();
16 +           InstructionCounter[OpcodeName]++;
17 +       }
18 +   }
19     //***** ASSIGNMENT END
    *****
20
```

Practice with Code Agent

매 수정 사항마다 검토 후 실행 권한을 부여
스스로 build / IR Generation / Pass 적용 / 결과 분석을 하는 것을 확인할 수
있음

```
lab3 > llvm-structure > test.ll
1 ; ModuleID = 'test.c'
2 source_filename = "test.c"
3 target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64"
4 target triple = "x86_64-pc-linux-gnu"
5
6 @.str = private unnamed_addr constant [2 x i8] c"*\00", a
7 @.str.1 = private unnamed_addr constant [2 x i8] c"*\0A\00", a
8
9 ; Function Attrs: noinline nounwind uwtable
10 define dso_local i32 @main() #0 {
11 entry:
12   %retval = alloca i32, align 4
13   %x = alloca i32, align 4
14   %y = alloca i32, align 4
15   %i = alloca i32, align 4
16   %j = alloca i32, align 4
17   store i32 0, ptr %retval, align 4
18   store i32 12, ptr %x, align 4
19   store i32 20, ptr %y, align 4
20   store i32 0, ptr %j, align 4
21   br label %for.cond
22
23 for.cond:                                ; preds = %entry
24   %0 = load i32, ptr %j, align 4
25   %cmp = icmp slt i32 %0, 20
26   br i1 %cmp, label %for.body, label %for.end8
27
28 for.body:                                ; preds = %for.cond
29   store i32 0, ptr %i, align 4
30   br label %for.cond1
31
32 for.cond1:                                ; preds = %for.body
```

✓ Shell Build the project and run the pass on test.ll to veri...

```
... 57 hidden (Ctrl+O) ...
add : 2
call : 2
ret : 1
load : 6
br : 10
alloca : 5
store : 7
icmp : 3
-----
```

```
lab3 > llvm-structure > test.out
1 Instruction hit counts
2 -----
3 add : 2
4 call : 2
5 ret : 1
6 load : 6
7 br : 10
8 alloca : 5
9 store : 7
10 icmp : 3
11 -----
12
```

Codex CLI 설치

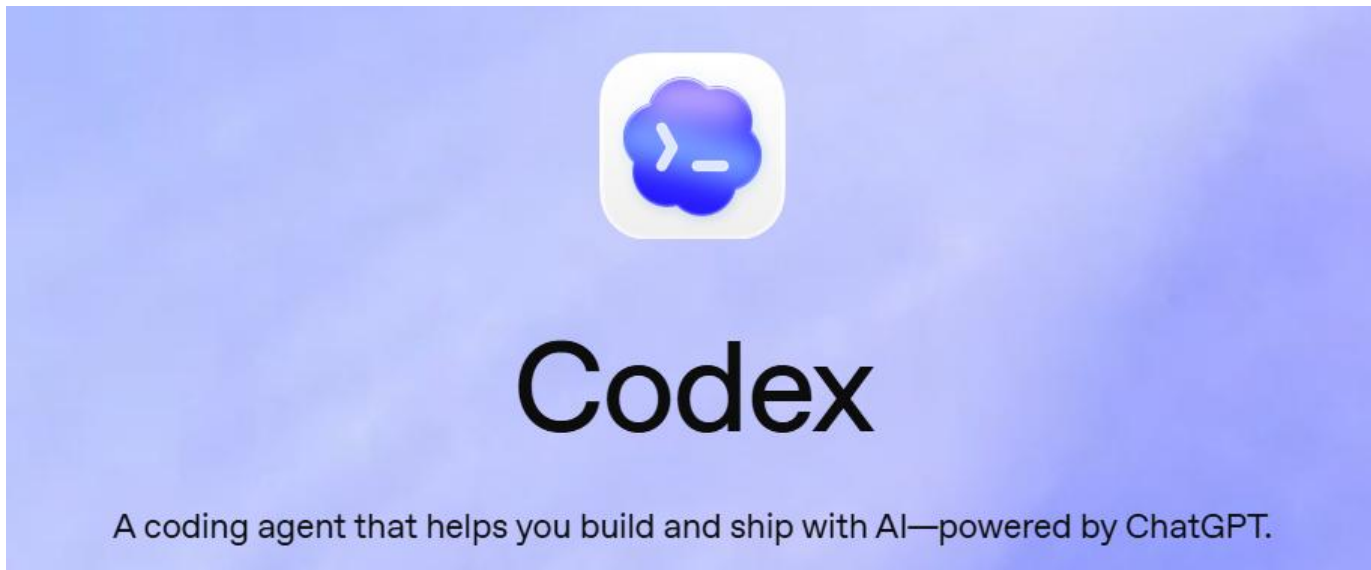
Open AI의 Coding Agent

> 설치 명령어

- `npm i -g @openai/codex`

> 실행 명령어

- Working Directory로 이동하여 실행
- `codex`



Codex CLI Commands

/ 를 입력하면 사용가능한 명령어들이 표시됨

```
>_ OpenAI Codex (v0.125.0)

model:      gpt-5.5  /model to change
directory:  /workspaces/system-expert-compiler-2026

Tip: GPT-5.5 is now available in Codex. It's our strongest agentic coding model
yet, built to reason through large codebases, check assumptions with tools, and
keep going until the work is done.

Learn more: https://openai.com/index/introducing-gpt-5-5/

• Unrecognized command '/help'. Type "/" for a list of supported commands.

> /
```

| | |
|---------------|--|
| /model | choose what model and reasoning effort to use |
| /fast | toggle Fast mode to enable fastest inference with increased plan usage |
| /permissions | choose what Codex is allowed to do |
| /experimental | toggle experimental features |
| /memories | configure memory use and generation |
| /skills | use skills to improve how Codex performs specific tasks |
| /review | review my current changes and find issues |
| /rename | rename the current thread |

Codex CLI Commands

/model

- 사용할 모델과 reasoning effort 설정 가능

```
Select Model and Effort
Access legacy models by running codex -m <model_name> or in your config.toml

1. gpt-5.5 (current) Frontier model for complex coding, research, and real-
   world work.
2. gpt-5.4 Strong model for everyday coding.
3. gpt-5.4-mini Small, fast, and cost-efficient model for simpler coding
   tasks.
4. gpt-5.3-codex Coding-optimized model.
> 5. gpt-5.2 Optimized for professional work and long-running agents.

Press enter to select reasoning effort, or esc to dismiss
```

```
Select Reasoning Level for gpt-5.2

1. Low Balances speed with some reasoning; useful for
   straightforward queries and short explanations
> 2. Medium (default) Provides a solid balance of reasoning depth and latency for
   general-purpose tasks
3. High Maximizes reasoning depth for complex or ambiguous problems
4. Extra high Extra high reasoning for complex problems

Press enter to confirm, or esc to go back
```

Codex CLI Commands

/status

- 현재 모델과 남은 사용량을 표시함

/init

- Gemini와 유사

`/status`

```
>_ OpenAI Codex (v0.125.0)
```

```
Visit https://chatgpt.com/codex/settings/usage for up-to-date  
information on rate limits and credits
```

```
Model:          gpt-5.2 (reasoning medium, summaries auto)  
Directory:      /workspaces/system-expert-compiler-2026  
Permissions:    Custom (workspace-write, on-request)  
Agents.md:      <none>  
Account:        donguk.red@gmail.com (Free)  
Collaboration mode: Default  
Session:        019dd562-9574-7311-85c4-def942a0efbe
```

```
Weekly limit:   [██████████░░░░░░░░░░] 61% left (resets 16:38 on 5 May)
```



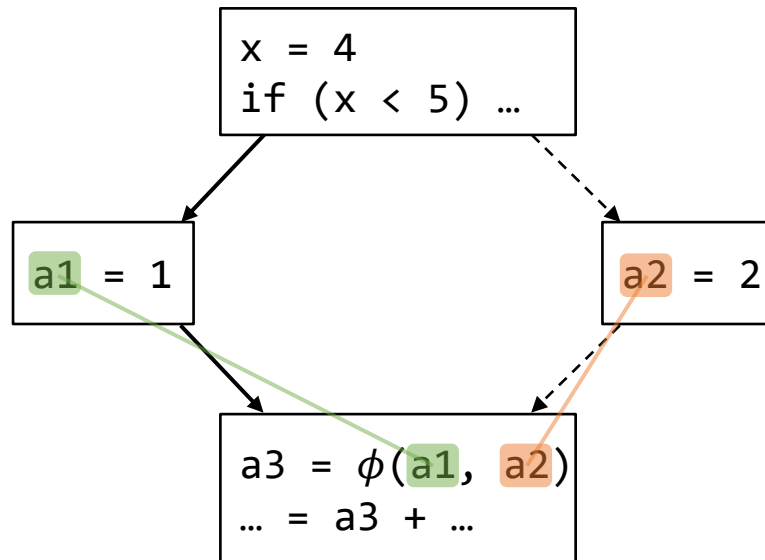
SCCP

SCCP

Sparse Conditional Constant Propagation (1991)

➤ Conditional Constant

- 계산 가능한 조건 (conditional, predicate) 들을 반영한, control-flow를 분석
⇒ 더 세밀한 분석
- e.g.,



- 동적 실행 흐름 (dynamic control-flow) 을 고려하면 `a3`도 constant

Value Substitution

Three-address form

- 하나의 변수가 여러 번 define 되어 특정 시점의 변수 값을 정확히 알기 어려움
 - e.g. x, y 는 모두 상수지만 a 의 def 가 여러 번 존재하여 값 파악 힘들

SSA

- SSA에서는 하나의 변수에 단 하나의 def 만 존재함
- 어떤 변수의 값을 알아내기 유리
 - e.g. a_0, a_1 의 값이 분리되어 x, y 의 값을 쉽게 상수로 치환 가능

LLVM IR: SSA form

```
a = 4
x = a + 2
a = 7
y = a + 2
```

Three-address form

```
a0 = 4
x = a0 + 2
a1 = 7
y = a1 + 2
```

SSA form



PROJECT 4

Project 4

SimpleSCCPTransform

- 정수 값과 일부 사칙 연산을 분석하고 변수 값을 상수로 치환 가능한 SCCP, SimpleSCCP를 구현하시오

input

- `mem2reg` pass를 통과하고 난 이후의 LLVM IR
 - Stack 에 `alloca()` 로 할당된 메모리를 register 로 승급(promotion)시켜주는 pass

output

- Constant value substitution 이 적용된 LLVM IR

Objective

- 주어진 test.c에 SimpleSCCP를 적용하고, 결과를 예시 출력과 비교한다.
- SCCP가 적용될 수 있는 새로운 testcase.c를 직접 작성한다.
- 보고서 작성
 - 구현체가 어떻게 동작하는지 코드 리뷰 작성
 - testcase.c에 SimpleSCCP를 적용하고 분석 결과 정리

C++ 문법 및 LLVM API 숙지 부담을 줄이기 위해 Vibe Coding을 적극적으로 활용할 것

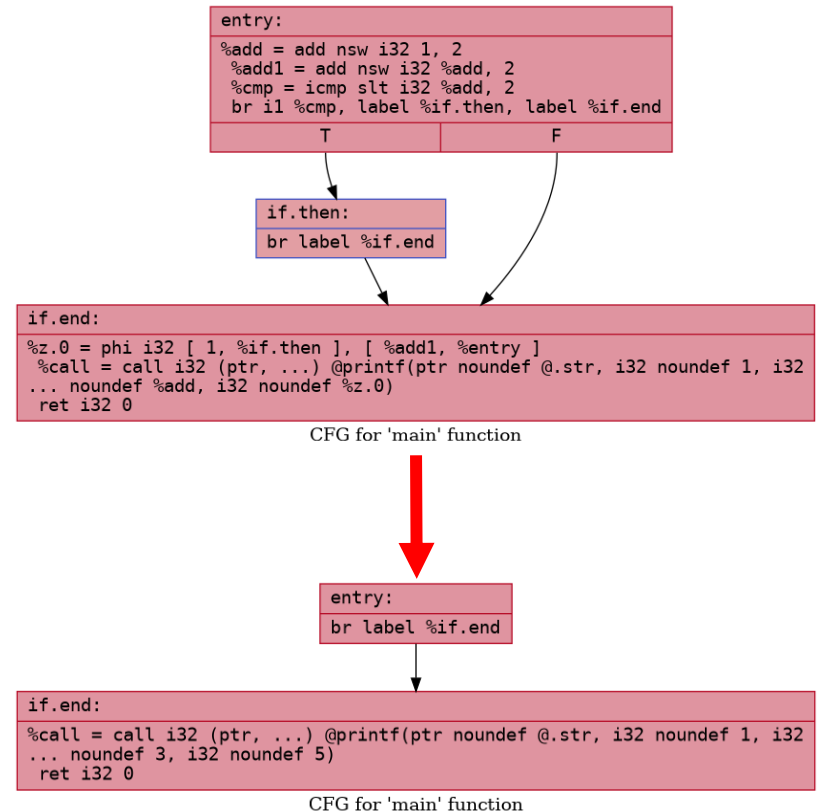
SCCP Transform Implementation

Example

- %cmp 의 값이 compile-time 에 알려짐
- 무의미한 operation 과 branch 가 제거됨

Workflow

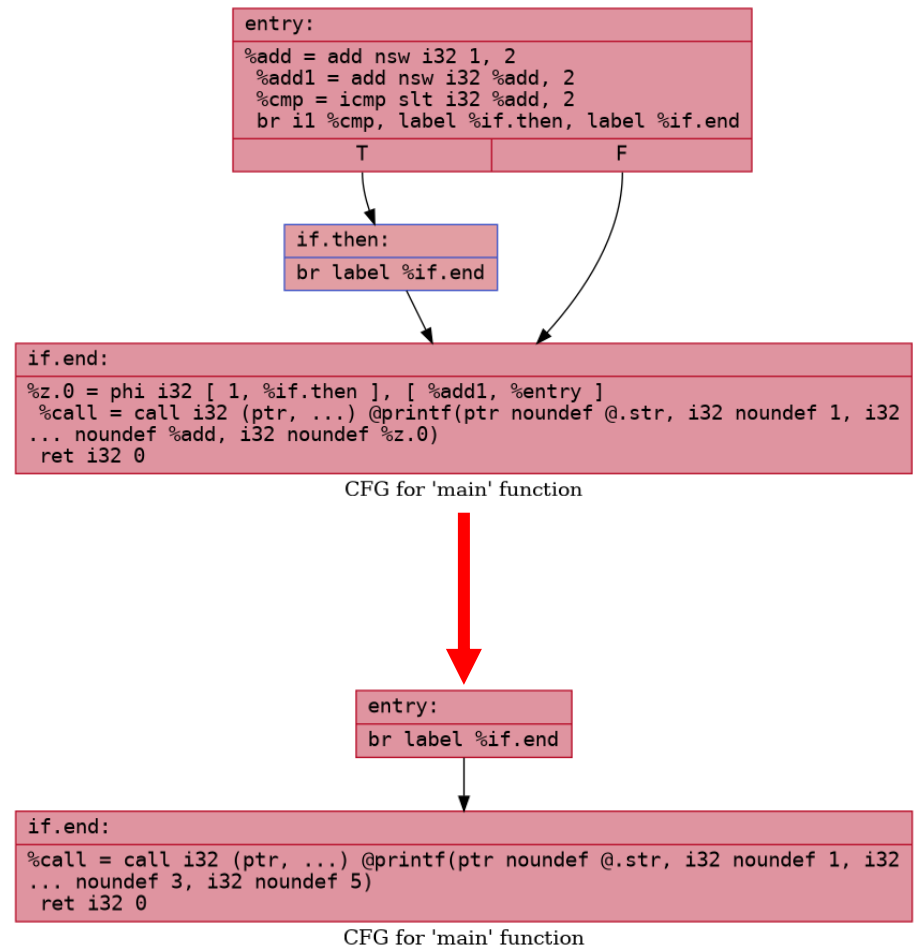
- 상수로 분석된 Instruction 제거 및 치환
- Branch Instruction의 condition이 상수인 경우 unconditional branch로 변환
 - 접근되지 않는 block은 제거
- Phi Instruction의 경우 도달 불가능한 block을 부터 오는 값은 제거



SCCP Transform Implementation

Example 1

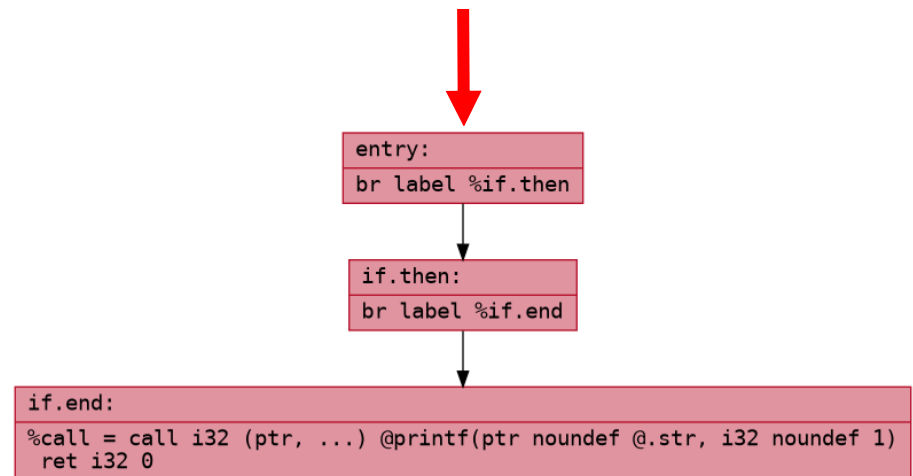
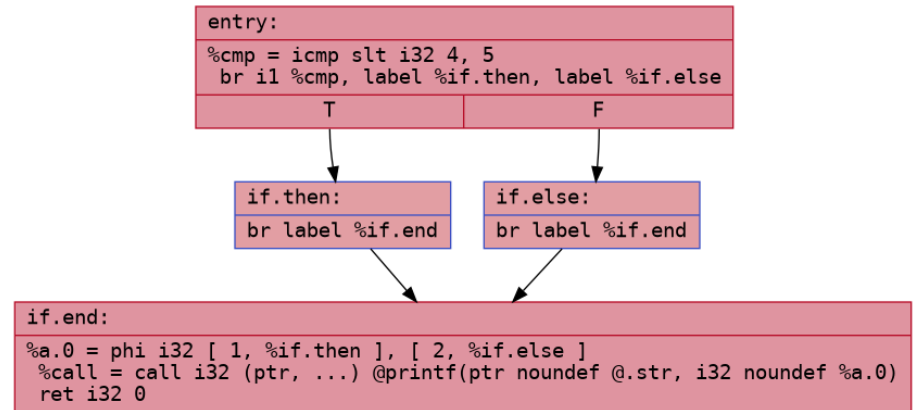
```
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = 1;
6      int y = x + 2; // 3
7      int z = y + 2; // 5
8
9      if (y < 2) // false
10     {
11         z = 1;
12     }
13
14     // x = 1, y = 3, z = 5
15     printf("%d %d %d\n", x, y, z);
16
17     return 0;
18 }
```



SCCP Transform Implementation

Example 2

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a;
6      int x = 4;
7
8      if (x < 5)
9      {
10         a = 1;
11     }
12     else
13     {
14         a = 2;
15     }
16
17     printf("%d\n", a);
18
19     return 0;
20 }
21
```



Useful Commands

Build

- `cmake -S . -G Ninja -B build`
- `cmake --build build`

Generate IR from the source.

- `clang -O0 -S -emit-llvm -Xclang -disable-O0-optnone -Xclang -disable-llvm-passes -fno-discard-value-names test.c -o test.ll`

Run 'mem2reg' pass.

- `opt -S -passes='mem2reg' ./test.ll -o input.ll`

Call your pass from opt.

- `opt -S -load-pass-plugin build/lib/libSimpleSCCP.so -passes='simple-sccp' ./input.ll -o out.ll`

Run 'verify'.

- `opt -S -passes='verify' out.ll`

Run 'dot-cfg' pass

- `opt -passes='dot-cfg' out.ll -disable-output`

DOT to png Conversion

- `dot -Tpng .main.dot -o test.png`